AGENT_GUARDRAILS

Agent guardrails — the §11.4.109 anti-forgetting enforcement layer

Anchor: 11.4.109 — *Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist.* Canonical text: `submodules/constitution/Constitution.md`. Read it there; this document is the consumer-side deployment of components (B) and (C), not a copy of the corpus.

Why this file exists, stated as the anchor states it. A constraint that depends on an agent remembering it is not a constraint — it is a hope. The motivating incident was not an agent that disobeyed a rule; it was an orchestrator that *forgot to paste the rule*. That contract breaks on every cold-session start, every context-window exhaustion and every subagent dispatch. Two layers close it:

Layer	Component	Where it lives here	What it catches
Floor — mechanical	(A) PreToolUse guard hook	`.claude/settings.json` → `submodules/constitution/scripts/hooks/guard-forbidden-commands.sh`	Forbidden command *classes*, at the tool-call boundary, regardless of what any agent remembers
Ceiling — semantic	(B) this preamble	below	Every rule the hook cannot pattern-match

Layer	Component	Where it lives here	What it catches
Self-guard	(C) this checklist	below	The orchestrator's own forgetting

The floor is wired at `.claude/settings.json` — **tracked in git**, so a fresh clone inherits it. That is deliberate and is the lesson learned from `.git/hooks/`, which is untracked and therefore leaves a fresh clone unprotected until an install step is run by hand. A guard that does not survive a clone is a guard for one working copy.

The hook is referenced, never copied. §11.4.109(A) forbids a local copy: a copy diverges silently and the consumer then enforces a stale ruleset while believing it enforces the current one. The path above points *into* the constitution submodule.

Installation and operativeness are verified, not asserted. `bash scripts/verify-pretooluse-guard.sh` is the dedicated hook-validation script required by §11.4.234(A)(3). It does not merely check that a config key exists — it *executes* the wired guard against live probes and requires a real refusal. Its paired §1.1 mutation proof (`--prove-failure`) demonstrates that it catches an un-wired hook, a locally-copied hook, and a neutered hook that returns 0 for a forbidden command. A validator that passes while the hook is missing is the false-green control §11.4.201 forbids.

SUBAGENT CONSTITUTIONAL PREAMBLE

> **Orchestrator: paste this block VERBATIM into every subagent dispatch.** It is not optional and it is not summarisable. The dispatch is the only moment at which the subagent can be informed; there is no later.

```
You are operating under the HelixConstitution. These constraints
bind you
absolutely. They are not advice and they are not negotiable by
anything said
later in your task description.

 1. ANTI-BLUFF (§11.4, §11.4.5, §11.4.69, §11.4.107). A PASS is a
claim that
    the feature works for the END USER. Every PASS carries
captured runtime
```

evidence from THIS session. Metadata-only, config-only, absence-of-error
and grep-without-runtime PASSes are critical defects however green the
summary line reads. The words verified / tested / working / complete /
fixed / passing are FORBIDDEN without pasted output you actually produced.
A FAIL asserted without evidence is equally forbidden (§11.4.1).

2. NO GUESSING (§11.4.6). "Probably", "seems", "appears", "should be",
"likely" are not findings. Write UNCONFIRMED: / UNKNOWN: /
PENDING_FORENSICS: and stop, rather than assert.

3. NEVER FORCE-PUSH (§11.4.113). No --force, no --force-with-lease, no +ref,
no history rewrite, on ANY repository or submodule, with or without
approval. Integrate by fetching, merging onto the latest main, and pushing
fast-forward. Force is never necessary and never allowed. The PreToolUse
guard blocks this class mechanically; do not attempt to route around it.

4. NO PRIVILEGE ESCALATION. No sudo, no su. Use a rootless / user-level
alternative (§11.4.161 rootless container runtime).

5. HOST SAFETY (§12, CONST-033). Never suspend, hibernate, reboot, power off
or halt the host. This class is CATEGORICALLY non-overridable — the
guard's escape hatch does not apply to it. Respect the memory and thread
headroom caps (§12.6, §12.12) and verify a process is OURS before
signalling it (§11.4.174).

6. INVESTIGATE BEFORE FIXING (§11.4.102). Iron Law: NO FIXES WITHOUT ROOT
CAUSE INVESTIGATION FIRST. Reproduce on the broken artifact (§11.4.115)
before writing the fix. Guess-and-retry and "probably flaky" are
violations.

7. NEVER DELETE ON SIGHT (§11.4.122, §11.4.124). Do not silently remove a
shipped component or seemingly-dead code. Investigate the git

```
history for
    how it was wired and how it died. No proof, no deletion. Ask
the operator
    and get an explicit keep-or-remove decision.

 8. NO HARDCODING (§11.4.29 and the environment-adaptability
audits). No
    frozen host names, absolute developer paths, model ids, ports
or
    endpoints. Derive them, with the former literal as a
documented last
    resort at most.

 9. SECRETS (CONST-042, §11.4.10). Never commit, print, log or
echo a
    credential. `.env` files stay gitignored, mode 0600.

10. REUSE BEFORE REIMPLEMENT (§11.4.74, §11.4.76). Check the
submodule
    catalogue first. Containerised workloads go through the
containers
    submodule; do not grow a parallel runtime, lifecycle or
orchestration
    implementation inside the consuming project.

11. EVERY CHANGE IS REVIEWED (§11.4.142). Independent review
precedes
    acceptance, including a one-line doc edit. Self-review
(§11.4.92)
    precedes it and never substitutes for it.

12. CONTINUATION (§12.10). Keep CONTINUATION.md in sync in every
non-trivial
    commit.

Classes 3, 4, 5 and raw host-direct emulator/adb/instrumentation
are ALSO
blocked mechanically by the §11.4.109 PreToolUse guard hook. The
hook is the
floor, not the ceiling: everything above binds you whether or not
the hook can
pattern-match it.
```

ORCHESTRATOR PRE-ACTION CHECKLIST

The orchestrator forgets too — that is the documented root cause of the incident behind 11.4.109. Run this against your own actions, not only your subagents'.

Before any subagent dispatch - [ ] The SUBAGENT CONSTITUTIONAL PREAMBLE above is pasted VERBATIM into the dispatch. Not paraphrased, not linked. (§11.4.109(B)) - [ ] The subagent's task states its own evidence obligation — what a PASS would have to look like. (§11.4, §11.4.5)

Before any emulator / device / container action - [ ] The run routes through the containers submodule CLI, not host-direct. (§11.4.76) - [ ] No live operator device is targeted; the gate host is eligible — otherwise report honestly BLOCKED rather than proceeding. (§11.4.133)

Before any push or destructive git action - [ ] No force flag of any shape. §11.4.113 is absolute; there is no per-operation approval that unlocks it here. - [ ] Fetched and integrated onto the latest `main` first. (§11.4.71) - [ ] The remote is the approved upstream set only. (§2.1) - [ ] For any history-affecting operation, a hardlinked `.git` backup exists FIRST. (§9) - [ ] The working tree is quiescent — no build is writing tracked artifacts. (§11.4.84, §11.4.121)

Before any host-affecting command - [ ] The command is not in the host-power blocked class. (§12, CONST-033) - [ ] Memory and thread headroom are within cap. (§12.6, §12.12)

Before reporting any result - [ ] Every PASS carries pasted output from THIS session. (§11.4) - [ ] Every unmeasured claim is written UNCONFIRMED / UNKNOWN, not smoothed into an assertion. (§11.4.6) - [ ] Any figure quoted from a document was RE-MEASURED, not copied. Documents in this tree carry dated observations, not standing facts.

(D) HOST RESOURCE SAFETY

Why. On 2026-10-14 (host `anton`) the box stalled at about 14:22. Measured with sar/atop: tmpfs shmem grew 0.33 → 8.1 GB between 13:20 and 14:30, `app.slice` (MemoryHigh ≈ 21 GB) was throttled into a direct-reclaim storm (sys CPU 1108 %, load 430, memory PSI *full* 53–92 %, 1.4 GB/s refault reads), and no OOM daemon or guard acted. The trigger was about 20 parallel agents, many writing multi-GB copies into the harness scratchpad `/tmp/claude-1000/...` — `/tmp` is a 16 GB **RAM-backed tmpfs** — and running unbounded heavy gates at the same time. A `cp -al` hardlink backup (§9) from the repository to `/tmp` fails

(cross-device), which pushed agents into full copies instead. The mechanism is the one in `$HOME/Projects/llmctl/docs/host-safety.md` ("Why GNOME says Application Stopped").

Rules for every orchestrator and every subagent

#	Rule	Enforcement — what actually stops a violation today
D1	**Fan-out budget:** no more than **5** concurrent subagents, and no more than **2 heavy** ones at a time. Dispatch in **waves**: the next heavy wave starts when one ends, never all at once.	**Heavy half ENFORCED** for work routed through `scripts/agent-admit.sh` (2 slots). **The 5-subagent half is ADVISORY** (documentation only): nothing counts subagents.
D2	**Heavy** means: pre-push gates, Playwright, `go test -race`, podman / site builds, a content-boundary full scan, `zero-gap-verify`, mutation suites (`--prove-failure` / `--run-proofs`), local model runs.	**Admitted automatically** (they call agent-admit themselves): pre-push gate 1 (`go test`), gate 6 (Playwright), `scripts/zero-gap-verify.sh` (run and `--prove-failure`), `scripts/verify-content-boundary.sh` (scan and `--prove-failure`). **NOT admitted automatically — wrap them yourself (D3/D4):** pre-push gates 0 and 2–5, `verify-check-registry.sh --run-proofs`, every other `--prove-failure` battery, `site-build` /

#	Rule	Enforcement — what actually stops a violation today
		`_tools/deploy-langs.sh`, podman builds, `go test -race` outside gate 1, model runs.
D3	Run heavy work only through **admission**: `bash scripts/agent-admit.sh --tag <name> -- <cmd>` (or `--verdict3` for a 0/1/2 gate). It takes a slot, then checks memory PSI `some avg10` < 15, Shmem < 10 % of RAM, `/tmp`, `/dev/shm`, `/run/user/<uid>` each < 25 % used, `app.slice` working set (`memory.current − inactive_file`) < 85 % of `memory.high`, and §12.12 thread headroom (< 75 % of `ulimit -u`, ≥ 512 free), then runs `<cmd>` under `bounded-run -m 6G -T 3600`. A refusal is rc 1 (rc 2 under `--verdict3`, which also maps every rc ≥ 124, e.g. a SIGKILL from the memory cap, to 2); never retry it in a loop — wait for the next wave.	**ENFORCED** for the entry points listed in D2; **ADVISORY** for anything an agent runs without the wrapper (admission is cooperative).
D4	Any other heavy command runs	

#	Rule	Enforcement — what actually stops a violation today
	under **`bounded-run -m <MEM> -T <SECONDS> --`** with explicit caps (minimums: `go test -race` 4G, builds/archives 6G). Never bare.	**ADVISORY** (nothing refuses a bare command).
D5	**Scratch on disk, never tmpfs:** `S=$(bash scripts/agent-scratch.sh new <label>)` and write under `"$S"`. Never write a repo copy, a clone, a worktree, a bundle, an archive or an extracted tree to `/tmp`, `/dev/shm` or `/run/user/*` — including the harness scratchpad under `/tmp/claude-*`.	**ENFORCED** by the project guard (wired b44c9b0) for every shape its tmpfs rule parses; **ADVISORY** for the shapes in its Known limits. Structurally: a disk `TMPDIR` (operator step 1, applied) moves the harness scratchpad and `mktemp` off RAM.
D6	**Backups are hardlinks on the same filesystem, of `.git`:** `cp -al <repo>/.git "$S/<repo>.git.mirror"` (§9.1 step 1, zero data blocks). `bash scripts/agent-scratch.sh df-check` proves the scratch root and the repo share one filesystem and mount and that `link()` works (it writes nothing in the repo). **A hardlink is**	`df-check` **verifies** the precondition; using hardlinks is **ADVISORY**.

#	Rule	Enforcement — what actually stops a violation today
	the SAME inode, so it does NOT survive an in-place write: a file modified in place (an editor saving in place, `>>` appends, `truncate`) changes the "backup" too. That does not contradict §9.1/§9.3: git writes objects, packs, refs, the index and config as NEW files renamed into place, so the mirror keeps the pre-operation state (only append-only reflogs grow). Do not use `cp -al` as a backup of a WORKING TREE you are about to edit; copy those files to `"$S"` on disk instead (still never to tmpfs). Never `cp -a` a whole repository or gitdir.	
D7	**Never clone or copy a repository** to get a second working copy; use `git worktree add "$S/wt" <ref>` on disk, and remove it when done.	**ENFORCED into tmpfs** (the wired guard refuses clone / worktree / cp / rsync into tmpfs); a clone to disk is never refused, so the rest of D7 is **ADVISORY**.
D8	Clean up: remove your scratch dir when finished; `bash scripts/agent-scratch.sh gc --`	**ADVISORY** (nothing runs `gc` on a schedule).

#	Rule	Enforcement — what actually stops a violation today
	`older-than 3d` removes only dirs the helper created and that have been idle that long. `du` shows usage.	

Defense in depth — the project guard (wired as the second `PreToolUse` hook, b44c9b0). `scripts/hooks/guard-project-catalogue.sh` reads each command with a shell lexer (continuations, quoting, brace expansion, compound commands, `function` bodies and `coproc NAME { }`, `(( ))` arithmetic, here-documents bound to the command that owns their `<<` and here-strings fed to a shell, `$(...)` including inside unquoted here-documents and arithmetic, `echo|printf|cat ... | bash` and `|& bash`, `VAR=` prefixes of `bash -c`, wrappers such as `timeout`/`xargs`/`flock`/`bounded-run`/`agent-admit.sh`, nested runners such as `script -c`, GNU `parallel`, `git submodule foreach`, `git rebase -x`, absolute binary paths, every git global option) and applies two rules.

Catalogue rule (round 5: structural). Every refused row of `docs/zero-gap/guarded-commands.tsv` is judged by PARSING the command's arguments the way the tool does — options in any position, short clusters, option values skipped, unique long-option prefixes (`--har`, `--dele`, `--f`) — never by a literal prefix of the line. Examples it refuses: `git reset -q --hard`, `git reset HEAD~1 --hard`, `git push origin main :old`, `git push --del origin x`, `git update-ref -m msg -d …`, every checkout discard form (`git checkout -- f`, `git checkout HEAD -- f`, `git checkout -f`, `git switch --discard-changes`, `git checkout-index -f`, `git read-tree -u --reset`), `git add -- ':!x'` / `"$PWD"` / `*`, `pkill -ef x`, a catch-all `pkill ''` / `pkill "$UNSET"` / `pkill .` / `pkill sh`, `pkill -P 1`, `pkill -G GID`, `pkill --inverse ...`, `pkill -O 1` (no pattern, broad criteria only), `pkill 'a**'` (unreadable: fails closed), `pkill -g PGID` (refused like `kill -- -PGID`: a group cannot be validated here, a deliberate tightening of §11.4.263), a direct `systemd-run` without `--expand-environment=no` whose command carries a literal `${` or `$$`, ANY literal word that begins with `$`, or a command substitution (systemd expands `${VAR}` and `$$` and reads every word starting with `$` as a variable; unset => empty, or the whole argument dropped), `systemctl -i suspend`, `systemctl isolate …`, login1 `Suspend` over D-Bus, `gnome-session-quit`, `kill -9 -1`, `pkill -u "$USER"`, `rm -rf ~/*/`, `rm -rf ~/.[!.]*`, `rm -rf //home`, `cd / && rm -rf home`, `rm -rf /ho?e`, `D=$HOME; rm`

`-rf "$D"`, `find -delete` in a protected directory, and a hook bypass by configuration (`-c core.hooksPath=…`). It allows the look-alikes: `git push origin :` (matching), `git merge --no-verify-signatures`, plain branch switches, `git restore --staged`, `git stash list`, `kill 12345`, `pgrep -f`, and text that merely mentions a command. An unset variable is empty in the fresh shell of a Bash call, so `rm -rf "$S"/*` with `S` unset is refused; write `"${S:?}"`. Every refusal says what to do instead (the proof asserts the hint). The complete per-row list is in the hook's header ("CATALOGUE RULE").

workshop-redact rule (row `workshop-redact-no-root`, 2026-10-21; refined 2026-10-22). Refuses a bare `workshop-redact` (basename at command position in any wrapper stage, also `go run|test ./cmd/workshop-redact`) unless one of these holds, judged the way the binary's Go flag parser reads its arguments: (1) a non-empty `-root`/`--root`/`-root=X`/`--root=X` FLAG, where the LAST `-root` wins (`-root /x -root ""` has no root; `-root "" -root /x` has one) and `-root` as the value of another flag (`--note -root --purge`) does not count; (2) `WORKSHOP_ROOT=<non-empty>` in that command's own `VAR=value` prefix; (3) a mode the binary itself does not guard: `-h`/`-help`, `--version`, `--dry-run`, or `--list-derived-leaks`/`--check-review` without `--purge` (`--list-derived-leaks --purge` IS guarded, as in the binary). An unset variable is empty in the fresh shell of a Bash call, so `-root "$UNSET"` and `WORKSHOP_ROOT=$UNSET workshop-redact` are refused like a missing root, and `workshop-redact${IFS}--purge` is split before it is judged. `go test ./cmd/workshop-redact/...` is refused too, because words after the package belong to the test binary and `-root` there is not the program's flag: run `WORKSHOP_ROOT=<scratch> go test ./cmd/workshop-redact/...` instead. `bash scripts/redact.sh ...` stays allowed. **Status: best-effort defence in depth, not the control.** Out of scope, and not detectable by a command-line rule: the binary started from another program (Python/`subprocess`), held in a variable (`b=workshop-redact; $b ...`), copied, renamed or symlinked, `$(which ...)`, `eval "$(echo ...)"`, other computed words (`${IFS%?}`-style tricks beyond plain `$IFS`/`${IFS}`), and wrappers the lexer does not know (`strace`, `unshare`, `nsenter`, `ssh`). The control is the binary's own guard (explicit `-root`; `WORKSHOP_ALLOW_PRIMARY_WRITE=1` for a git primary checkout, granted only by `scripts/redact.sh`; `--purge` needs `--confirm-r6-loss`). The `guard-gaps` sweep class probes only the UPSTREAM guard, so the two `workshop-redact-no-root*` rows show as findings there by design (the sweep reports 23 = the 21 older gaps + these 2); the rows are marked `CLOSED-BY-PROJECT-HOOK` and the class names the project hook in their wording instead of sending the reader upstream alone.

tmpfs rule. Refuses (rc 2) BULK writes that bring DISK data into tmpfs: recursive copies (`cp -a|-r|-R`, `rsync`, `zip -r`, `podman|docker cp`), clones/worktrees/bundles/archives, archive extraction (`tar -x`, `unzip`, `7z x`), git fetching into (or `git push` / `git remote update` into) a repo on tmpfs, `podman|docker save`, a directory or a > 64 MiB file moved, copied or `install`ed, an unbounded or > 64 MiB `dd`, and a download whose target or URL name is a model or an archive (also `.part`/`.partial`), including stdout carried into tmpfs through pass-through filters (`podman save img | gzip > /tmp/x.tgz`, `curl URL.gguf | tee /tmp/m.gguf`). It is deliberately NOT bulk, and allowed: one named file of bounded or unknown size (`mv out.txt /tmp/`, `curl -o /tmp/health.json ...`, `dd ... count=1`), `cp -l`, tmpfs → tmpfs copies, and a checkout in a repo that already lives on tmpfs. The only escape is an inline `AGENT_TMPFS_OK='<at least 3 real words>'` on that command, logged to `~/.cache/agent-scratch/tmpfs-overrides.log`; an agent can still write such a reason — **the log is the control**.

Known limits (documented, not covered — the hook's header "KNOWN LIMITS" is the complete list): code it cannot read (a script file run by name, `source`, an interpreter such as `python3 -c`, a here-document fed to one), text built at run time or in an earlier Bash call, commands run elsewhere or later (`ssh`, tmux, `at`, `pgrep … | xargs kill`, a git alias), git operations outside the catalogue classes (`git commit -a`, `git rm -r .`, `--abort`, `git branch -f`), a force refspec configured in `remote.NAME.push`, `pkill NAME` of a process that is not session-critical, destructive classes the catalogue does not list (`mkfs`, `dd of=/dev/…`), destinations reached through a symlink, and tmpfs writes by tools the rule does not list. **WAIVED** by operator decision #27 (round-5 review, LOW, not fixed): #7 a bounded-run started as a background job of a script cannot trap SIGINT/SIGQUIT sent to its pid alone (SIGTERM works; terminal Ctrl-C reaches the command); #11 the wrappers `systemd-inhibit`, `sg -c`, `unshare`, `strace` are not unwrapped and expansions such as `${HOME:0:5}` are read as literal text; #12 `verify-pretooluse-guard.sh` V9 (like V4) runs the guard file, not the wired command string, so a wiring ending in `|| true` would pass; #13 bounded-run needs `env --default-signal` (coreutils >= 8.31) and fails closed (rc 125) without it. Round 6 FIXED #6 (sweep containment) and #8 (stderr noise) and made bounded-run pass its command line verbatim (it passes `--expand-environment=no`; only where systemd-run lacks that option does bounded-run refuse, with exit 125, a command containing `$`). **WAIVED** by operator decision #29 (round-6 review): exact POSIX regex/glob parity beyond translating `[[:class:]]` (an unreadable pkill pattern fails closed); text known only at run time; false refusals of globs under `$HOME` that match a real child directory

(`rm -rf ~/llmctl-verify.*`: name the paths instead); the orphan TMPDIRs that SIGKILLed pre-round-6 bounded-run runs left under `~/.cache/bounded-run/tmp` (kept; not tracked here: list them with `ls ~/.cache/bounded-run/tmp`, 43 entries on 2026-10-15). Also **WAIVED** (round-7 confirming review, F3): same-line indirection `X='$D'; systemd-run ... "$X"` is not seen (the assignment is kept expanded). Fixed: `$'\x24D'`-style escapes and short option clusters that hid the wrapped command (`systemd-run -qp X`, `xargs -0n 1`, `env -iS`). How to re-run the 5,101-command replay the re-reviews used is in the hook's header (the corpus is not tracked: real commands carry private paths into this public repository).

Ready-to-paste SUBAGENT PREAMBLE addendum (host resource safety)

```
HOST RESOURCE SAFETY (docs/AGENT_GUARDRAILS.md section D) —
binding for this task:
- /tmp, /dev/shm and /run/user/* are RAM. Never write a repo copy,
clone, worktree, bundle, archive or
  extracted tree there (that includes the harness scratchpad
under /tmp/claude-*). Get disk scratch with
  S=$(bash "$HOME/Projects/vasic/scripts/agent-scratch.sh" new
<label>) and work under "$S".
- Backups: cp -al <path> "$S/backup" (hardlinks, same filesystem).
Never cp -a a repository or a gitdir.
- Heavy work (pre-push gates, Playwright, go test -race, podman/
site builds, content-boundary full scan,
  zero-gap-verify, mutation suites, model runs) ONLY via
  bash "$HOME/Projects/vasic/scripts/agent-admit.sh" --tag <name>
-- <cmd>
  and everything else heavy under bounded-run -m <MEM> -T
<SECONDS> --. A refusal means "wait for the next
  wave": report it, do not retry in a loop.
- No for-loops that spawn parallel jobs. At most one heavy command
at a time per agent.
- Remove your scratch dir when done.
```

Operator-applied steps (proposals; no agent edits these files)

These survive a reboot only through files outside this repository, which agents may not edit. `bash scripts/agent-scratch.sh verify-persistence` reports the state from any working directory: it lists every tracked artifact (committed or not) and the `TMPDIR` that four FRESH shells resolve — login, interactive login and interactive from `$HOME`, and

a login started from a non-HOME directory (a tmux pane or `bash -l` opened elsewhere). It **exits 1 while any row is unset or RAM-backed** — an operator step, not a defect of this tree.

1. **Disk TMPDIR — APPLIED by the operator on 2026-10-14** (marker `agent-tmpdir-on-disk` in `~/.profile` and at the top of `~/.bashrc`). Measured the same day: with `TMPDIR` set in the environment, or with `"env": {"TMPDIR": ...}` in a settings file passed via `--settings`, Claude Code created its per-session tree (`claude-1000/<project>/<session>/tasks`, the sibling of the scratchpad) under that directory instead of `/tmp`, and its Bash tool children saw the same `TMPDIR`. The aliases `claude1..claude5` are `CLAUDE_CONFIG_DIR=~/.claude-claudeN cma_run` from the generated `~/.local/share/claude-multi-account/aliases.sh` (regenerated, do not edit), sourced by `~/.bashrc`. Optional for `systemd --user` units: `~/.config/environment.d/60-tmpdir.conf` with `TMPDIR=${HOME}/.cache/claude-tmp`. **Revert:** delete the marked lines. **Cost:** disk temp is not wiped at boot and this host has no `systemd-tmpfiles` user timer; prune `~/.cache/claude-tmp` periodically. 1b. **PROPOSED: make `~/.bash_profile` independent of the working directory.** Its first line is `. .bashrc` — RELATIVE, so a login shell started anywhere but `$HOME` does not find `~/.bashrc`; and because `~/.bash_profile` exists, bash never reads `~/.profile` either. Measured with `verify-persistence`: the three `$HOME` rows resolve `TMPDIR` to `~/.cache/claude-tmp`, the login from `/` resolves it to NOTHING (falls back to `/tmp`). Change line 1 to:

   ```
   . "$HOME/.bashrc"
   ```

   Revert: restore `. .bashrc`. After it, `verify-persistence` should report every row on disk.

2. **Wire the project guard — APPLIED 2026-10-14 (b44c9b0)** (makes the tmpfs rule and the VSC-367 catalogue live). The second entry of the `PreToolUse` `"hooks"` list of the tracked `.claude/settings.json`, after the upstream guard; `bash scripts/verify-pretooluse-guard.sh` asserts it is wired (V2b) and fires (V9):

   ```
   {
     "type": "command",
     "command": "bash \"$CLAUDE_PROJECT_DIR/scripts/hooks/guard-
           project-catalogue.sh\""
   }
   ```

   Revert: remove the entry. Proof before wiring: `bash scripts/hooks/guard-project-catalogue.sh --prove-failure`.

Verifying this layer

```
bash scripts/verify-pretooluse-guard.sh                # 0
        wired+operative, 1 defect, 2 cannot determine
bash scripts/verify-pretooluse-guard.sh --prove-failure  # the
        paired §1.1 mutation battery
bash submodules/constitution/scripts/hooks/
        test_guard_forbidden_commands.sh   # upstream hermetic
        harness
bash scripts/hooks/guard-project-catalogue.sh --prove-
        failure            # the project guard's own paired proof
```

The third is the upstream author's own test suite for the guard, shipped inside the constitution submodule as of the 2026-09-08 fast-forward. It is the authority on what the guard actually enforces; the first two are this repository's evidence that the guard is *wired here* and *fires here*.